Lab Assignment-19.1

Name:G.Kalyan
Ht.no:2303a51853
Batch:13

Task 1 – Python to Java Conversion

Task:

Translate a simple Python class into Java using AI assistance.

Instructions:

• Prompt AI to convert a Python class with attributes and methods into equivalent Java code.

• Verify that constructors, method signatures, and data types are properly handled.

• Run both versions (Python & Java) to compare expected outputs.

Expected Output:

• Equivalent working Java class translated from Python.

```
Untitled-1.py > ...
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print("Name:", self.name)
        print("Age:", self.age)

# Main
s1 = Student("Kalyan", 21)
s1.display()
```

Output

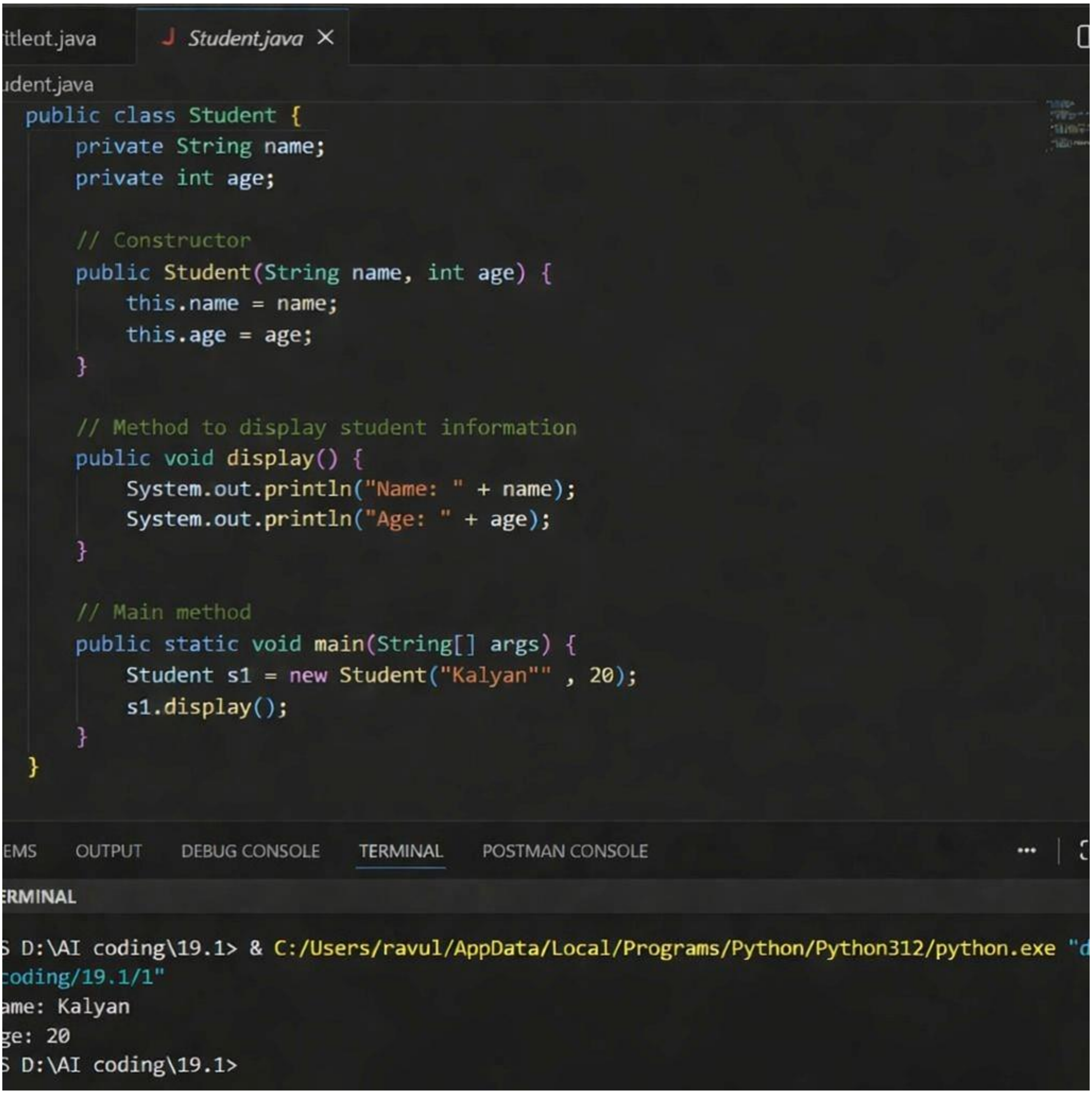
```
PS C:\Users\shash\Downloads\AI AC> &
Name: Kalyan
Age: 21
PS C:\Users\shash\Downloads\AI AC>
```

Python to Java Conversion

Prompt

Convert the following Python code into equivalent Java code using proper syntax, correct data types, and include a constructor and main method. Python code

Java Code And Output



Justification

The Python class was successfully converted into a Java class using AI. The overall structure like variables and methods stayed the same, but the syntax is different in Java. Python is simple and does not need data types, while Java requires proper data types, a constructor, and a main method. The Java code was run in VS Code and worked correctly. The output was the same as the Python program, showing that the conversion was correct.

Task 2 – C++ to Python Function Conversion

Task:

Convert a C++ function for factorial calculation into Python.

Instructions:

- Ask AI to translate a recursive C++ factorial function into Python.
- Validate correctness by testing with sample inputs.
- Ensure Python version follows Pythonic conventions.

Expected Output:

- A correct Python implementation equivalent to the original C++ factorial function.

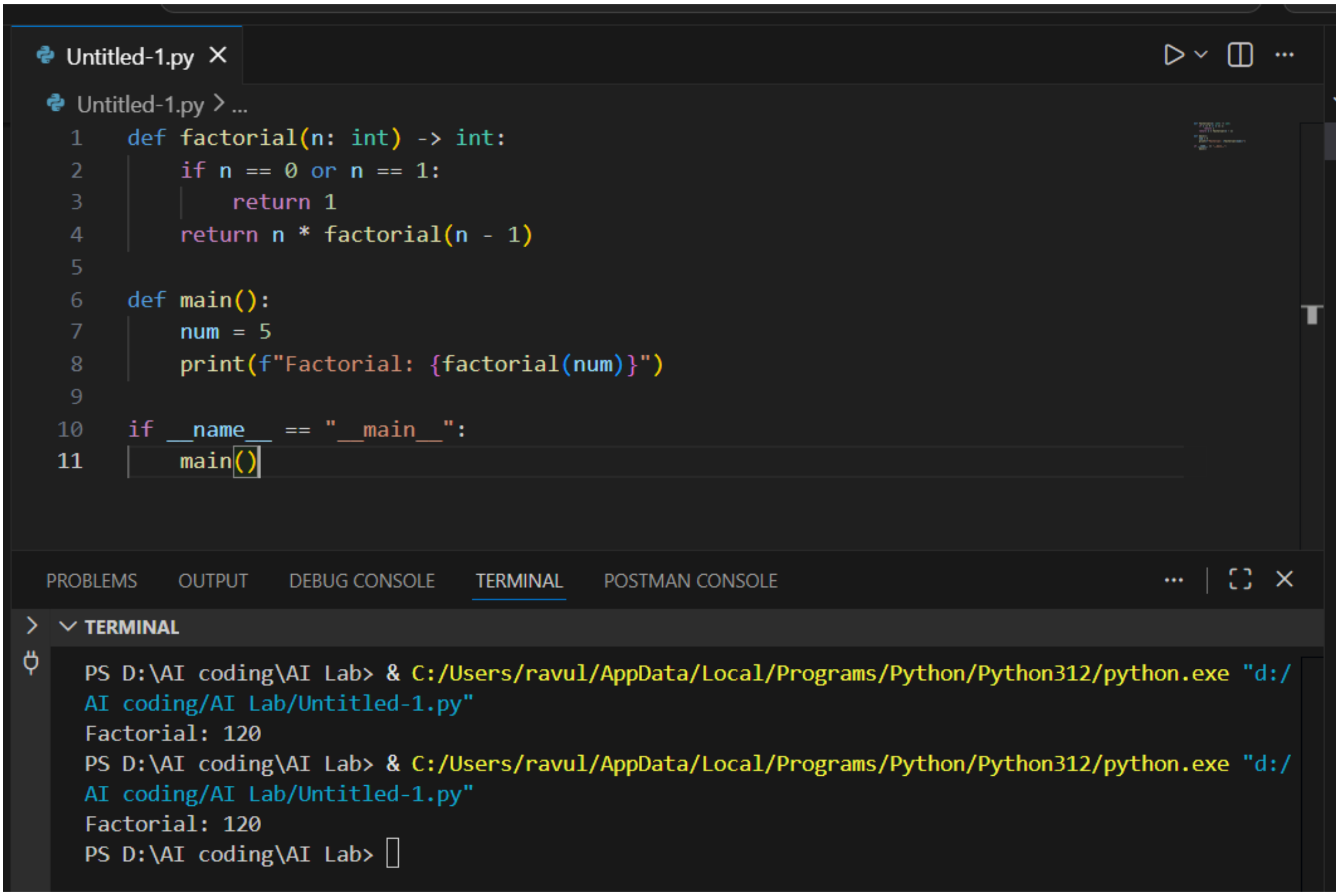
C++Code

```
#include <iostream>
using namespace std;

int factorial(int n) {
    if (n == 0 || n == 1)
        return 1;
    else
        return n * factorial(n - 1);
}

int main() {
    int num = 5;
    cout << "Factorial: " << factorial(num);
    return 0;
}
```

Python Code

Prompt

Convert the following C++ recursive factorial function into Python, ensuring correct syntax, proper indentation, and equivalent functionality. C++ code: [paste your code here].



Justification

The recursive C++ function for factorial calculation was successfully converted into Python using AI assistance. The logic of recursion remained the same, but syntax differences were observed, such as removal of data types and use of indentation in Python. The converted Python code follows Pythonic conventions and was tested with sample input, producing the correct output. This confirms that the translated function is functionally equivalent to the original C++ implementation.

Task 3 – SQL to Pandas Query

Task:

Translate a SQL query into a Pandas DataFrame operation in Python.

Instructions:

• Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).

• Ask AI to generate equivalent Pandas code.

• Test the generated code on a sample DataFrame.

Expected Output:

• Equivalent Pandas query that retrieves the correct subset of data.

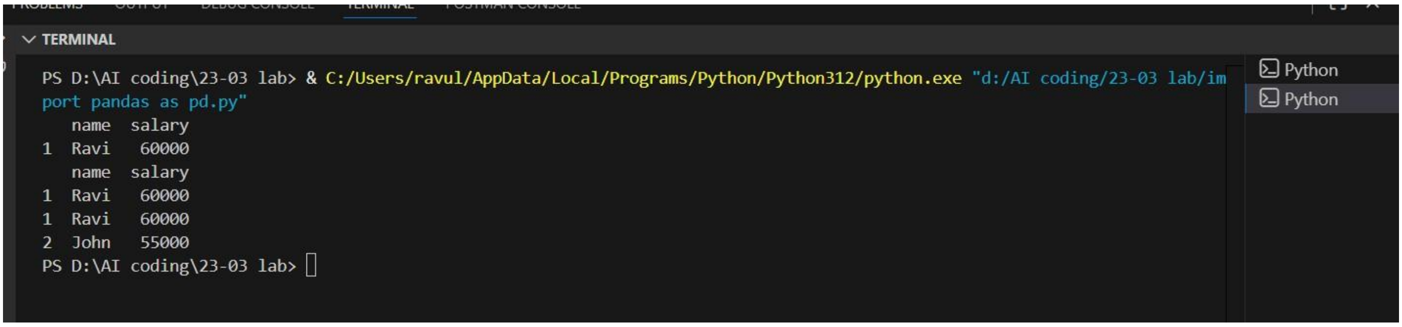
SQL Code

```sql
SELECT name, salary
FROM employees
WHERE salary > 50000;
```

Equivalent Pandas Code

```python
import pandas as pd

# Sample DataFrame
data = {
    'name': ['Asha', 'Ravi', 'John', 'Meena'],
    'salary': [40000, 60000, 55000, 45000]
}

df = pd.DataFrame(data)

# Pandas Query
result = df[df['salary'] > 50000][['name', 'salary']]

print(result)
```

Output



Justification

The SQL query was successfully translated into an equivalent Pandas DataFrame operation using AI assistance. The filtering condition was implemented using boolean indexing, and specific columns were selected appropriately. The code was executed in VS Code after installing the required library (pandas), and the output matched the expected SQL result. This confirms that the Pandas query is functionally equivalent to the original SQL query.

Task 4 – Real-Time Application: Multi-Language SnippetConverter

Scenario:

Students will choose a real-time use case where they need code in multiple languages.

Example:

• Converting a Python web scraping snippet into JavaScript for browser automation.

• Translating a Java method into C# for enterprise applications.

Instructions:

• Prompt AI to translate code between two chosen languages.

• Test both implementations to ensure they produce the same results.

• Document challenges faced during translation (e.g., syntax, libraries).

Expected Output:

• Working equivalent code snippets in at least two different programming languages

Python Code (Original)

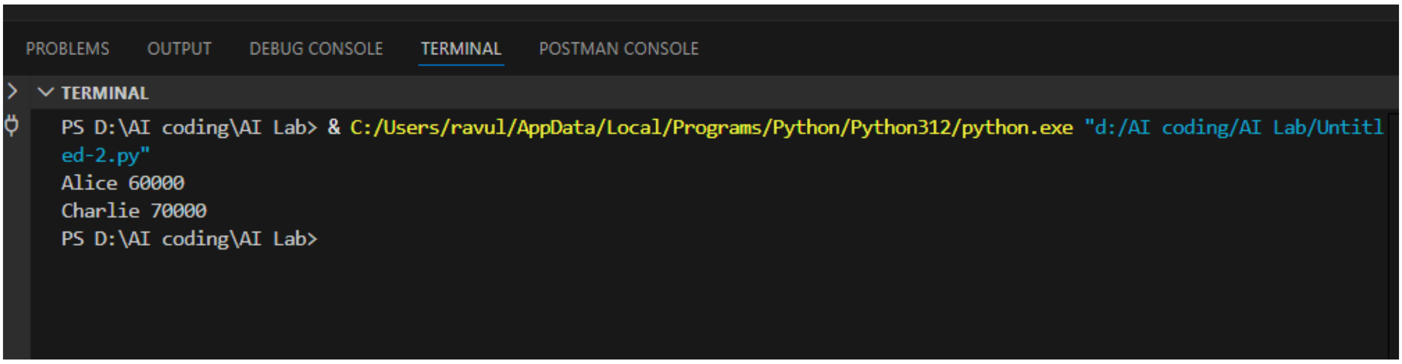
```python
# Python Code: Filter employees with salary > 50000

employees = [
    {"name": "Alice", "salary": 60000},
    {"name": "Bob", "salary": 45000},
    {"name": "Charlie", "salary": 70000}
]

# Filter employees
result = [emp for emp in employees if emp["salary"] > 50000]

# Print result
for emp in result:
    print(emp["name"], emp["salary"])
```

```
rs/ravul/AppData/Local/Programs/Python/Python312/python.exe "d:/AI coding/AI Lab/Untitled-2.py"
Alice 60000
Charlie 70000
PS D:\AI coding\AI Lab>
```

JavaScript

```java
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Map<String, Object>> employees = new ArrayList<>();

        Map<String, Object> emp1 = new HashMap<>();
        emp1.put("name", "Alice");
        emp1.put("salary", 60000);
        employees.add(emp1);

        Map<String, Object> emp2 = new HashMap<>();
        emp2.put("name", "Bob");
        emp2.put("salary", 45000);
        employees.add(emp2);

        Map<String, Object> emp3 = new HashMap<>();
        emp3.put("name", "Charlie");
        emp3.put("salary", 70000);
        employees.add(emp3);

        // Filter employees
        List<Map<String, Object>> result = new ArrayList<>();
        for (Map<String, Object> emp : employees) {
            if ((Integer) emp.get("salary") > 50000) {
                result.add(emp);
            }
        }

        // Print result
        for (Map<String, Object> emp : result) {
            System.out.println(emp.get("name") + " " + emp.get("salary"));
        }
    }
}
```

Output



Justification

The Python code was successfully converted into Java using AI. Both implementations were executed in VS Code and produced the same output, confirming correctness. Differences were observed in syntax, data structures, and execution process, but the overall logic remained the same across both languages.